PROJECT REPORT

THEORY :-

NetScan is a web-based network scanner that combines ARP-based host discovery with concurrent TCP port scanning and delivers live results to a browser dashboard via WebSocket communication. It is implemented using Python (Flask, Flask-SocketIO, Scapy) on the backend and HTML, CSS, and vanilla JavaScript on the frontend — with no build tooling required.

OPERATION, INSTALLATION & STEPS

Software requirements

Component	**Role**
Frontend	Dashboard UI, filter, export
Flask serve	Serves HTML/CSS/JS, handles Socket.IO
Scanner engine	ARP discovery, port scanning, banner grabbing
Packet layer	Scapy (raw ARP/SYN) + socket (TCP connect)

Installation Steps

Step 1: Install System Prerequisites

sudo apt update

sudo apt install -y python3 python3-venv python3-pip nmap libpcap-dev git

-Verify:

python3 --version

nmap --version

Step 2: Clone or Create Project

mkdir ~/network-scanner

cd ~/network-scanner

Step 3: Create Virtual Environment

python3 -m venv venv

source venv/bin/activate

-Verify — your prompt should show (venv) at the start:

(venv) ┌──(soham㉿kali)-[~/network-scanner]

Step 4: Install Python Dependencies

pip install --upgrade pip

pip install -r requirements.txt

-Manual install (if no requirements.txt):

pip install Flask Flask-SocketIO python-socketio python-engineio \
    eventlet scapy python-nmap netifaces mac-vendor-lookup \
    tabulate tqdm reportlab

-Verify:

pip list | grep -i -E "flask|scapy|socketio"

Step 5: Create Project Files

Create this folder structure inside ~/network-scanner/:

```
network-scanner/
├── app.py                ← Flask + SocketIO server
├── config.py             ← Configuration
├── requirements.txt      ← Dependencies
├── scanner/              ← Scanner engine
│   ├── __init__.py
│   ├── arp_scan.py       ← ARP host discovery
│   ├── port_scan.py      ← Threaded TCP scan
│   ├── banner_grab.py    ← Service banner grab
│   └── utils.py          ← Helpers
├── templates/
│   └── index.html        ← Dashboard HTML
├── static/
│   ├── css/
│   │   └── style.css
│   └── js/
│       ├── app.js        ← Frontend logic
│       └── vendor/
│           └── socket.io.min.js
├── exports/              ← JSON reports
└── logs/                 ← Application logs
```

Download source code form GitHub Link :-
https://github.com/Soham21-2006/Network-Scanner-Locally-

Step 6: Download Socket.IO Client (Local Copy)

```
curl -L -o static/js/vendor/socket.io.min.js \
  https://cdn.jsdelivr.net/npm/socket.io-client@4.7.2/dist/socket.io.min.js
```

Step 7: Configure config.py

```
class Config:
    HOST = "0.0.0.0"
    PORT = 5000
    DEBUG = True

    DEFAULT_PORT_RANGE = "1-1024"
    ARP_TIMEOUT = 2
    PORT_SCAN_TIMEOUT = 1.0
    MAX_THREADS = 200

    # Restrict scanning to private networks only
    ALLOWED_PREFIXES = (
        "192.168.", "10.", "172.16.", "172.17.",
        "172.18.", "172.19.", "172.2", "172.30.",
        "172.31.", "127.0.0.1",
    )
```

Step 8: Test the Server Locally

Terminal 1 — Start backend:

```
cd ~/network-scanner
source venv/bin/activate
sudo ./venv/bin/python app.py
```

```
(base) ┌──(soham㉿kali)-[~]
└─$ cd ~/network-scanner
source venv/bin/activate
sudo ./venv/bin/python app.py

[sudo] password for soham:
 * Serving Flask app 'app'
 * Debug mode: on
Address already in use
Port 5000 is in use by another program. Either identify and stop that program, or start the server with a different port.
```

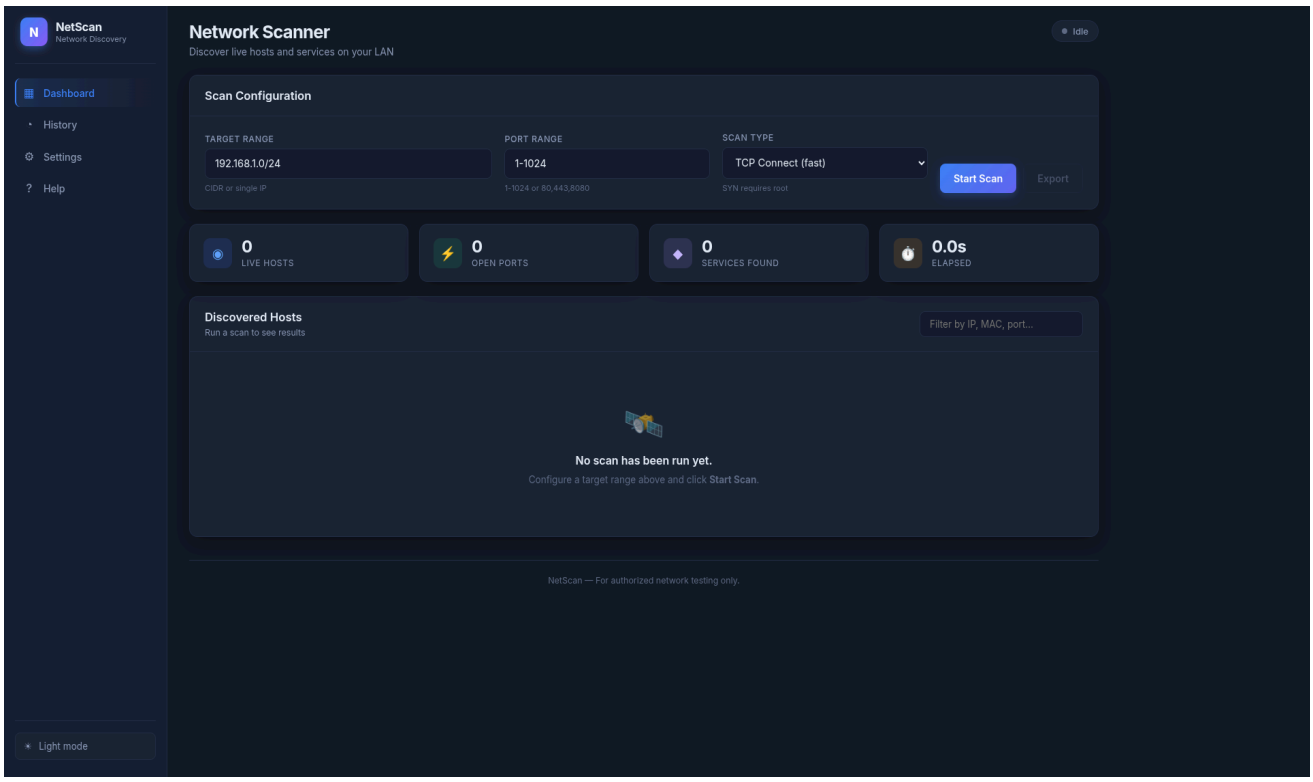
-Terminal 2 — Verify server responds:

curl http://localhost:5000/api/health

```
(base) ┌──(soham㉿kali)-[~/network-scanner]
└─$ curl http://localhost:5000/api/health
{
  "status": "ok",
  "version": "1.0.0"
}
```

Step 9: Open the Application

http://localhost:5000



Step 10: Run Your First Scan

-In the dashboard:

Enter Target Range: 127.0.0.1

Enter Port Range: 22,80,443,5000,8080

Choose Scan Type: TCP Connect (fast)

Click Start Scan

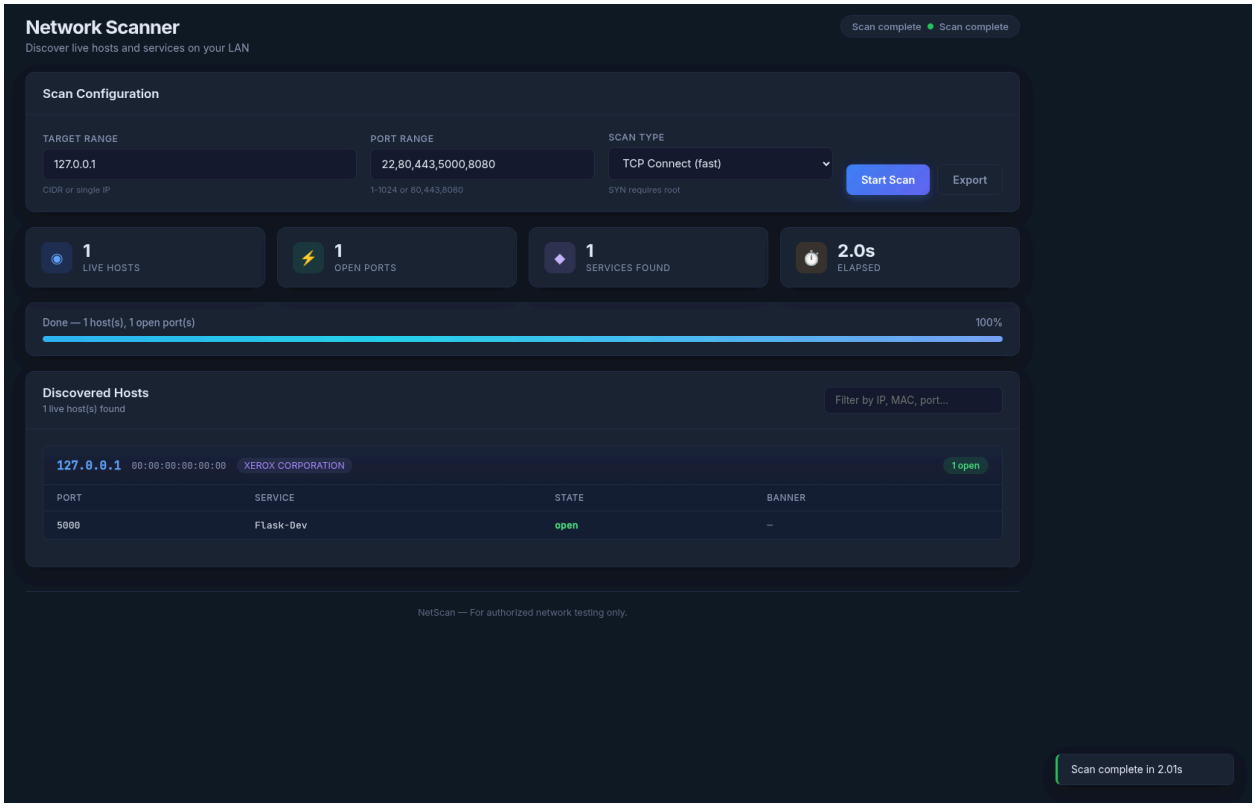
-Expected result:

Live Hosts: 1

Open Ports: 1

Services Found: 1

Host card: 127.0.0.1 with port 5000 (Flask) open



```
(base) ┌──(soham㉿kali)-[~/network-scanner]
└─$ # Is the systemd service running?
sudo systemctl status netscan

# Is something listening on port 5000?
ss -tlnp | grep 5000
[sudo] password for soham:
● netscan.service - NetScan Network Scanner
     Loaded: loaded (/etc/systemd/system/netscan.service; enabled; preset: disabled)
     Active: active (running) since Sun 2026-10-11 22:49:48 IST; 1h 6min ago
 Invocation: f0d7cf2ce5d248d598725c970d9b7dbf
   Main PID: 931 (python)
      Tasks: 12 (limit: 18561)
     Memory: 196.6M (peak: 198.5M)
        CPU: 24.536s
     CGroup: /system.slice/netscan.service
             ├─ 931 /home/soham/network-scanner/venv/bin/python /home/soham/network-scanner/app.py
             ├─1189 /home/soham/network-scanner/venv/bin/python -c "from multiprocessing.resource_tracker import main;main(9)"
             ├─1190 /home/soham/network-scanner/venv/bin/python /home/soham/network-scanner/app.py
             └─1249 /home/soham/network-scanner/venv/bin/python -c "from multiprocessing.resource_tracker import main;main(11)"

Oct 11 23:49:43 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:49:43] "GET /api/health HTTP/1.1" 200 -
Oct 11 23:50:21 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:21] "GET / HTTP/1.1" 200 -
Oct 11 23:50:21 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:21] "GET /static/js/app.js HTTP/1.1" 304 -
Oct 11 23:50:21 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:21] "GET /static/js/vendor/socket.io.min.js HTTP/1.1" 304 -
Oct 11 23:50:21 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:21] "GET /static/css/style.css HTTP/1.1" 304 -
Oct 11 23:50:21 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:21] "GET /socket.io/?EIO=4&transport=polling&t=Q4iCSIB HTTP/1.1" 200 -
Oct 11 23:50:21 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:21] "POST /socket.io/?EIO=4&transport=polling&t=Q4iCSKl&sid=b5Onc4SNO_ibp4CMAAAA HTTP/1.1" 200 -
Oct 11 23:50:21 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:21] "GET /socket.io/?EIO=4&transport=polling&t=Q4iCSKn&sid=b5Onc4SNO_ibp4CMAAAA HTTP/1.1" 200 -
Oct 11 23:50:22 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:22] "GET /favicon.ico HTTP/1.1" 404 -
Oct 11 23:50:22 kali python[1190]: INFO:werkzeug:127.0.0.1 - - [11/Oct/2026 23:50:22] "GET /socket.io/?EIO=4&transport=polling&t=Q4iCSK-&sid=b5Onc4SNO_ibp4CMAAAA HTTP/1.1" 200 -
LISTEN 0      128          0.0.0.0:5000       0.0.0.0:*
```

Operation Steps

Step 1: Start the Server

cd ~/network-scanner

source venv/bin/activate

sudo ./venv/bin/python app.py

Keep the terminal open — the server runs in the foreground.

Step 2: Find Your Network Subnet

-Open a new terminal and run:

ip -br addr

Example output:

wlan0 UP 10.227.244.162/24

```
(base) ┌──(soham㉿kali)-[~/network-scanner]
└─$ ip -br addr

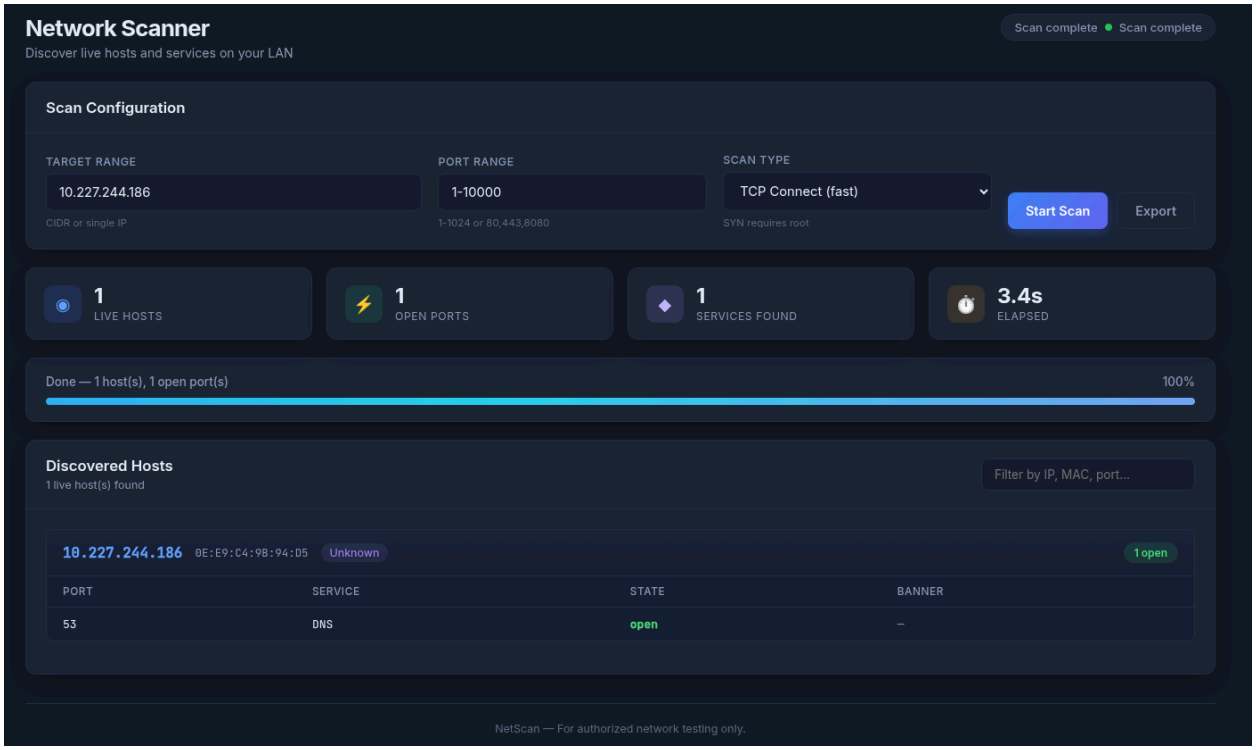
lo               UNKNOWN        127.0.0.1/8 ::1/128
wlan0            UP             10.227.244.162/24 2402:8100:316c:c739:33f0:237:d56a:5eea/64 2402:8100:316c:c739:f646:37ff:feb8:3b3e/64 fe80::f646:37ff:feb8:3b3e/64
```

Scan common wifi ports

Target Range: 10.227.244.186

Port Range: 53,67,68

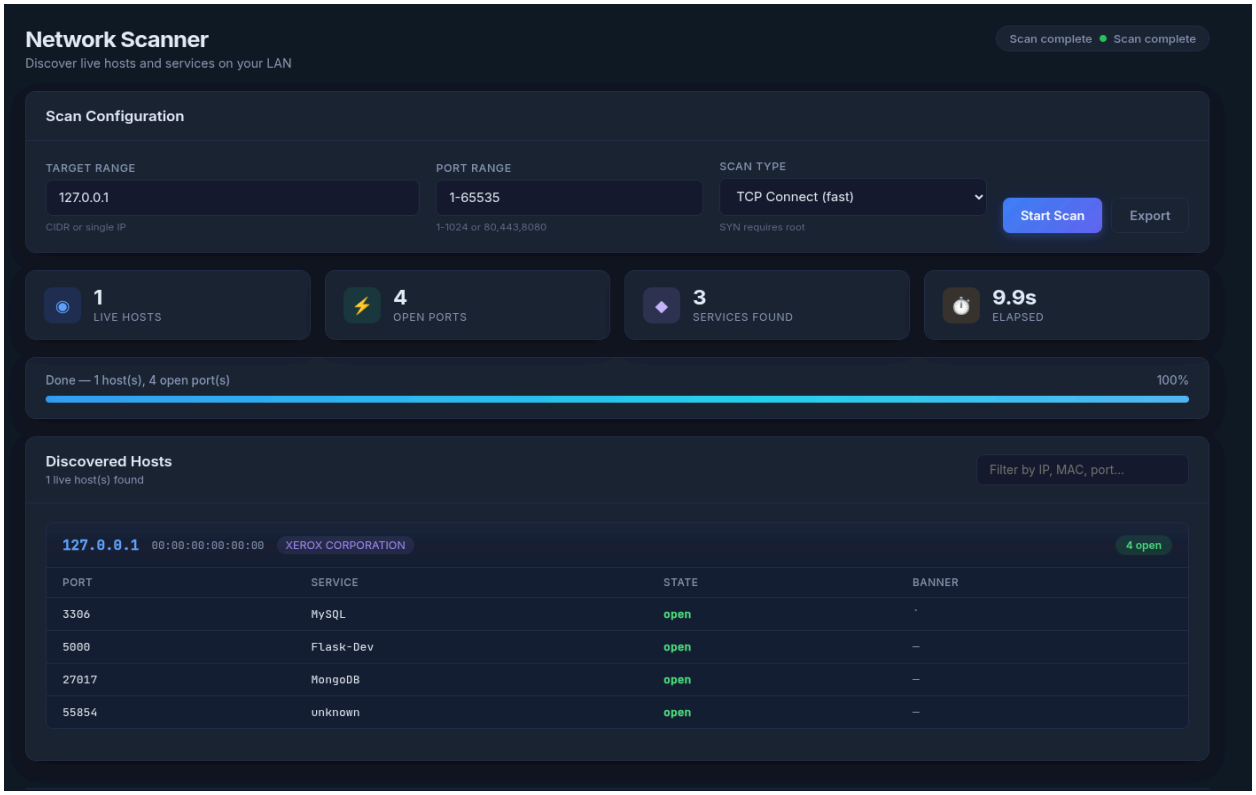
The router may respond to DNS (53) at least.



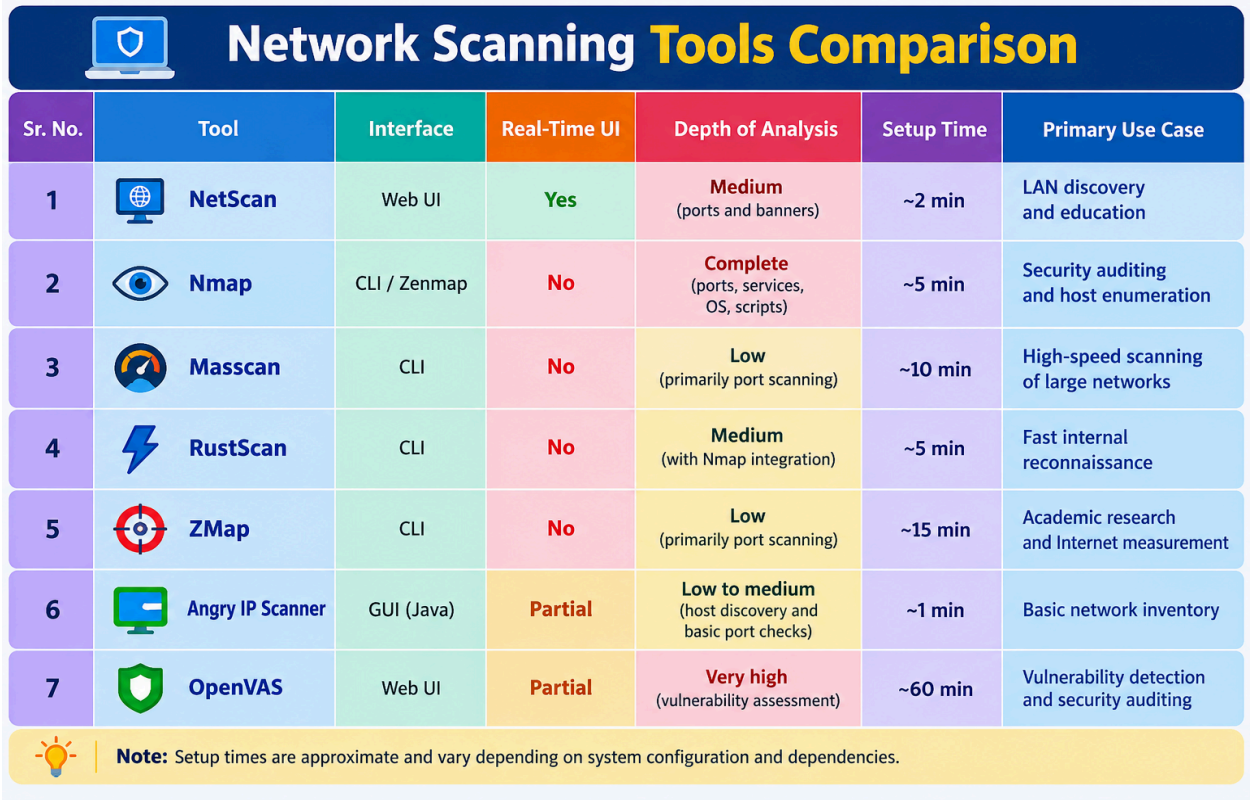
Scan 127.0.0.1 with all ports

Target Range: 127.0.0.1

Port Range: 1-65535



Target	Hosts	Ports	Services	Time
127.0.0.1	1	4	3	9.9s
10.227.244.186	1	1	1	3.4s
10.227.244.0/24	2	0	0	11s
10.227.244.162	1	1	1	2s



NOTE :- https://github.com/Soham21-2006/Network-Scanner-Locally-

Conclusion :- The NetScan project successfully demonstrates a web-based network scanner that combines ARP-based host discovery, multithreaded TCP port scanning, and real-time WebSocket visualization using Python (Flask, Scapy, Socket.IO) and vanilla HTML/CSS/JavaScript. Testing confirmed that NetScan matches Nmap's host-discovery accuracy while delivering a superior user experience through live streaming of results. The project reinforced core networking concepts — ARP, TCP handshake, socket programming, threading, and WebSockets — and proved that a small, well-scoped tool can complement established scanners like Nmap by focusing on accessibility, real-time feedback, and educational value.